

DOC-02

VIXOR ENGINEERING BIBLE

# كتاب البنية البرمجية

الوثيقة الرسمية لبنية نظام فيكسور الهندسية

# جدول المحتويات

مبادئ البنية

هيكل المجلدات

حدود النطاقات

مسؤوليات الطبقات

قواعد الاعتماديات

الطبقة المشتركة

طبقة الميزات

طبقة التطبيق

طبقة البنية التحتية

اتفاقيات التسمية

تنظيم الكود

قواعد إدارة الحالة

معايير واجهات البرمجة

معايير الخدمات

معايير المكونات

معايير الهوكس

معايير المزودات

معالجة الأخطاء

التسجيل والمراقبة

قواعد الأداء

إرشادات التوسع

قواعد المؤسسة

## مبادئ البنية

تستند بنية نظام فيكسور الهندسية إلى مجموعة من المبادئ الأساسية التي تضمن الاتساق والقابلية للصيانة والتوسع على المدى المبادئ ليست مجرد إرشادات عامة، بل قواعد صارمة يلتزم بها كل مطور يعمل على المشروع. تم صياغتها بناءً على أفضل الممارسات هندسة البرمجيات الحديثة، مع تعديلات تناسب طبيعة سوق التداول والعملات الرقمية السريعة التغير.

### المبدأ الأول: عزل النطاقات (Domain Isolation)

كل وحدة نطاقية `domain module` في فيكسور مستقلة تمامًا عن غيرها من النطاقات. لا يحق لأي نطاق الوصول مباشرة إلى النطاقات الداخلية لنطاق آخر. يقتصر التواصل بين النطاقات على واجهات التصدير المحددة في ملف `index.ts` الرئيسي لكل نطاق. يضمن إمكانية تطوير أو استبدال أي نطاق دون التأثير على بقية النظام.

### المبدأ الثاني: الاعتمادية الأحادية (Unidirectional Dependencies)

يُبنى رسم الاعتماديات بين النطاقات على شكل رسم بياني موجه غير دوري **DAG**، مما يعني أن التبعية تسير في اتجاه واحد. حلقات معتمدة. هذا يضمن سهولة فهم تدفق البيانات ويمنع الاعتماديات الدائرية التي قد تؤدي إلى أخطاء يصعب تتبعها.

### المبدأ الثالث: الخادم أولاً (Server-First Architecture)

جميع العمليات الحساسة — مثل الوصول إلى قاعدة البيانات، واستدعاء واجهات التداول `ccxt`، والاتصال بمقدمي الذكاء الاصطناعي — تُنفذ حصريًا على الخادم. لا يُسمح للعميل بالوصول المباشر إلى قاعدة البيانات أو الخدمات الخارجية. هذا يحمي بيانات الاعتمادات والنظام.

### المبدأ الرابع: الأمان الصفري (Zero Trust Security)

يُعامل كل طلب وارد كأنه غير موثوق، حتى لو صدر من مستخدم مُصادق عليه. تُستخدم سياسات أمان الصفوف `RLS` في `se` جميع الجداول `27`، ويُتحقق من صلاحيات كل عملية على حدة. لا يُعتمد على مصادقة العميل فقط لحماية البيانات.

### المبدأ الخامس: النمطية القابلة للتجميع (Composable Modularity)

كل وحدة برمجية صُممت لتكون قابلة لإعادة الاستخدام والتكيب مع وحدات أخرى. يستخدم النظام نمط التجميع `barrel exports` في كل نطاق لتوفير واجهة نظيفة وموحدة. المكونات والهوكس والخدمات مصممة لتكون كتل بناء قابلة للتجميع بمرونة.

### المبدأ السادس: التسامح مع الأخطاء (Fault Tolerance)

يُبنى فيكسور نمط المرونة `Resilience Patterns` على مستوى الطبقة المشتركة، بما في ذلك قواطع الدوائر `Circuit Breakers` ومحددات المعدل `Rate Limiters`، وذاكرة التخزين المؤقت `LRU Cache`. عند فشل مزود ذكاء اصطناعي، ينتقل النظام إلى المزود التالي في سلسلة الاحتياط دون انقطاع الخدمة.

### المبدأ السابع: قابلية التوسع الأفقي (Horizontal Scalability)

صُمم النظام ليتوسع أفقيًا عبر استضافة بدون خوادم `Serverless` باستخدام Nitro 3.0 مع إعداد Vercel. تعتمد حالة التطبيق الخارجية مثل Upstash Redis بدلاً من الذاكرة المحلية للخادم، مما يسمح بتشغيل نسخ متعددة من التطبيق دون تعارضات.

### المبدأ الثامن: قابلية المراقبة (Observability)

كل عملية مهمة في النظام مُزودة بتتبع مُنظم عبر Sentry للأخطاء، و Mixpanel لتحليلات الاستخدام، ونظام أحداث مُطبّق بدقة 20 حدثًا مُطبّقًا. يُسجل كل خطأ مع سياقه الكامل لتمكين التشخيص السريع وحل المشكلات.

يدعم النظام العربية والإنجليزية بالكامل عبر نظام تدويل `i18n` مدمج في الطبقة المشتركة. جميع النصوص في واجهة المستخدم للترجمة، مع اتجاه تلقائي للصفحات حسب اللغة المختارة. يُحفظ اللغة كجزء من إعدادات المستخدم في قاعدة البيانات.

## 🔥 ملاحظة مهمة

هذه المبادئ التسعة تشكل الأساس الذي يُبنى عليه كل قرار هندسي في فيكسور. أي انتهاك لهذه المبادئ يتطلب موافقة كتابية من قائد الفريق. التقني مع تبرير واضح للأسباب.

SEC-02

## هيكل المجلدات

يتبنى فيكسور هيكل مجلدات صارم مبني على مبدأ الطبقات والنطاقات. فهرس المشروع الرئيسي `src/` ينقسم إلى أربع صيغ متتالية، كل طبقة بمسؤوليات محددة وواضحة. هذا الهيكل يضمن فصلًا واضحًا بين المسؤوليات ويجعل التنقل في المشروع للمطورين الجدد.

### الطبقة المشتركة — /shared

تحتوي على 96 ملفًا تُشكّل البنية التحتية المشتركة بين جميع النطاقات. تشمل هذه الطبقة وظائف الوصول إلى البيانات، ومُؤخّرات البيانات، ونظام التدويل، ونظام الأحداث، وذاكرة التطبيق، وسجل الأدوات، وبيانات السوق، ونظام المرونة، والإشعارات، والتشفير قابل للاستخدام من أي طبقة أعلى.

### طبقة الميزات — /features

تحتوي على 22 وحدة نطاقية `domain module`، كل منها يمثل مجال أعمال محدد. تشمل: التحليل `analysis`، والاختبار الاسترجاعي `arbitrage`، والوسيط `broker`، وذكاء الرسوم البيانية `chart-intelligence`، والبيانية `chart-truth`، والرفيق الذكي `copilot`، والحلقة اليومية `daily-loop`، والمناظرة `debate`، والاكتشاف واكتشاف العملات `discovery`، والتجارب `experiment`، والسوق `market`، وموكسي `moxi`، والملاحظات `notes`، والورقي `paper-trading`، وحاكم المخاطر `risk-governor`، وتتبع الإشارات `signal-tracking`، والاستراتيجيات والصفقات `trades`، والتداول `trading`، والمستخدم `user`، والمحفظة `wallet`، وقائمة المراقبة `watchlist`.

### طبقة التطبيق — /app

تحتوي على جميع المسارات `routes` والتخطيطات `layouts` والصفحات `pages`. يضم هذا المجلد 44 مسارًا متنوعًا، `AppShell` الذي يمثل الهيكل الرئيسي للتطبيق بواقع 1688 سطرًا برمجيًا، و `PageLayout` الذي يوفر التخطيط الموحد للمكونات. هذه الطبقة تجمع بين المكونات من طبقة الميزات وتُقدمها للمستخدم.

### طبقة البنية التحتية — /infrastructure

تحتوي على 14 نقطة نهاية `API endpoints` عبر Nitro، وإدارة الاتصال بـ Supabase مع 27 جدول بيانات، وتهيئة Redis، وإعداد نظام المراقبة عبر Sentry و Mixpanel. تُعزل هذه الطبقة تمامًا عن طبقة الميزات لضمان إمكانية تبديل أي خدمة خارجة على منطق الأعمال.

```
ed/ # الطبقة المشتركة (96 ملف)
data-access/ # وظائف الوصول إلى البيانات (26+ server fn)
LLM/ # موجّه نماذج اللغة (4 مزودات)
i18n/ # التدويل (en/ar)
events/ # نظام الأحداث (+20 حدث مُطَبّق)
memory/ # ذاكرة التطبيق (PostgreSQL-backed)
tool-registry/ # سجل الأدوات
market-data/ # بيانات السوق (BinanceWS, DexScreenerWS)
resilience/ # المرونة (circuit-breaker, rate-limiter, LRU)
notifications/ # الإشعارات (email/telegram/webhook/in-app)
crypto/ # التشفير (credential encryption)
ures/ # طبقة الميزات (22 نطاق)
analysis/ # التحليل (ملف 24، SMC/ICT ~9,924 سطر)
market/ # السوق (ورقة - الأكثر اعتمادًا)
trading/ # التداول (ملف 15)
wallet/ # المحفظة (ملف 16)
copilot/ # الرفيق الذكي (13 ملف، متعدد الوكلاء)
discovery/ # اكتشاف العملات (ملف 16)
... # نطاق إضافي 16
# طبقة التطبيق (44 مسار)
AppShell.tsx # الهيكل الرئيسي (1,688 سطر)
PageLayout.tsx # تخطيط الصفحات (929 سطر)
routes/ # المسارات والتخطيطات
rastructure/ # طبقة البنية التحتية
api/ # Nitro نقطة نهاية 14
database/ # Supabase (27 جدول)
integrations/ # Redis, Sentry, Mixpanel
```

#### 📁 قاعدة ذهبية للمجلدات

لا يجوز لأي ملف في طبقة التطبيق استيراد ملف من طبقة البنية التحتية مباشرة. يجب أن تمر جميع الطلبات عبر طبقة الميزات أو الطبقة

SEC - 03

## حدود النطاقات

يتألف نظام فيكسور من 22 وحدة نطاقية `domain module`، كل منها يمثل مجال أعمال مستقل بحدود واضحة ومحددة. النطاقات تُدار عبر رسم بياني موجه غير دوري **DAG**، مما يضمن عدم وجود اعتماديات دائرية ويجعل تدفق البيانات متنبئًا وسهل

### النطاقات الورقية مقابل النطاقات الجذرية

في الرسم البياني للاعتماديات، تُصنف النطاقات إلى نوعين: **النطاقات الورقية** `leaf domains` التي لا تعتمد عليها أي نطاقات أخرى، و**النطاقات الجذرية** `root domains` التي تعتمد عليها نطاقات متعددة. نطاق السوق `market` هو النطاق الورقي الأكثر النظام، حيث تُعتمد عليه نطاقات مثل التحليل والتداول والاكتشاف للحصول على بيانات الأسعار والأحجام.

| النطاق    | الوصف                           | عدد الملفات | الامتداد |
|-----------|---------------------------------|-------------|----------|
| analysis  | محرك التحليل الفني SMC/ICT      | 24          | ج        |
| arbitrage | محرك المراجعة بين المنصات       | 27          | ج        |
| backtest  | اختبار الاستراتيجيات استرجاعيًا | 9           | ج        |
| broker    | إدارة اتصال الوسيط              | 1           | و        |

| النطاق             | الوصف                                   | عدد الملفات |
|--------------------|-----------------------------------------|-------------|
| chart-intelligence | تحليل الرسوم البيانية بالذكاء الاصطناعي | 5           |
| chart-truth        | التحقق من دقة الرسوم البيانية           | 5           |
| copilot            | الرفيق الذكي متعدد الوكلاء              | 13          |
| daily-loop         | الحلقة اليومية للمهام المجدولة          | 3           |
| debate             | مناظرة خوارزمية بين الاستراتيجيات       | 7           |
| discover           | اكتشاف فرص جديدة                        | 2           |
| discovery          | محرك اكتشاف العملات الميمنية            | 16          |
| experiment         | إدارة التجارب والاختبارات               | 5           |
| market             | بيانات السوق والأسعار                   | 7           |
| moxi               | مساعد MOXI الذكي                        | 8           |
| notes              | ملاحظات المتداول                        | 3           |
| paper-trading      | التداول الورقي                          | 4           |
| risk-governor      | حاكم المخاطر                            | 4           |
| signal-tracking    | تتبع إشارات التداول                     | 3           |
| strategy           | إدارة الاستراتيجيات                     | 6           |
| trades             | سجل الصفقات                             | 3           |
| trading            | تنفيذ عمليات التداول                    | 15          |
| user               | إدارة المستخدمين                        | 5           |
| wallet             | إدارة المحافظ الرقمية                   | 16          |
| watchlist          | قائمة المراقبة                          | 3           |

## قواعد حدود النطاقات

- القاعدة 1:** لا يُسمح لملف في نطاق ما باستيراد أي شيء من مجلد داخلي لنطاق آخر. الاستيراد فقط عبر ملف `index.ts` الرئيس المستهدف.
- القاعدة 2:** لا يجوز لنطاقين من نفس المستوى في DAG الاستيراد من بعضهما البعض. يجب رفع الاعتماد المشترك إلى الطبقة الأعلى.
- القاعدة 3:** عند الحاجة إلى بيانات من نطاق ورقي، يتم استهلاكها عبر هوك مخصص في النطاق الطالب، لا عبر استيراد مباشر لملف الورقي.
- القاعدة 4:** يُحظر تمامًا إنشاء اعتماديات دائرية بين النطاقات. يتم فحص ذلك آليًا عبر أدوات تحليل الاعتماديات `dependency-cruiser`.

### ⚠ تحذير

أي محاولة لتجاوز قواعد حدود النطاقات ستُرفض في مراجعة الكود. استثناءات هذه القواعد تتطلب موافقة قائد الفريق التقني وإنشاء Issue الديون التقنية الناتجة.

## مسؤوليات الطبقات

يقسم فيكسور المشروع إلى أربع طبقات رئيسية، كل طبقة بمسؤوليات محددة وصارمة. هذا الفصل الواضح يضمن أن كل جزء من المشروع على مهمة واحدة فقط، مما يسهل الاختبار والصيانة والتوسع. فهم مسؤوليات كل طبقة هو الأساس للعمل بكفاءة في المشروع.

### طبقة الميزات — /features

تضم منطق الأعمال لكل مجال من مجالات النظام. كل نطاق على مكونات React، وهوكس مخصصة، وخدمات نطاقية، TypeScript، وأختبارات وحدة. تضم 22 وحدة نطاقية تشمل التحليل SMC/ICT بأكثر من 9,924 سطر، ومحرك المراجعة ومحرك اكتشاف العملات بـ 16 ملفًا، والرفيق الذكي متعدد ملفًا.

### الطبقة المشتركة — /shared

توفر البنية التحتية المشتركة لجميع الطبقات العليا. تضم وظائف الوصول إلى البيانات (26+ وظيفة خادم)، ومُوجّه نماذج اللغة الكبيرة مع 4 مزودات، ونظام التدويل ثنائي اللغة، ونظام الأحداث المُطَبَّق بدقة (20+ حدث)، وذاكرة التطبيق القائمة على PostgreSQL، وسجل الأدوات، وبيانات السوق الحية عبر BinanceWS وDexScreenerWS، ونظام المرونة، والإشعارات متعددة القنوات، وتشفير الاعتمادات. 96 ملفًا تشكل العمود الفقري للنظام.

### طبقة التطبيق — /app

المسؤولة عن تجميع المكونات من طبقة الميزات وتقديمها للمستخدم عبر 44 مسارًا. تحتوي على AppShell الرئيسي (1,688 سطر) الذي يُهيكل التطبيق بالكامل، وPageLayout (929 سطر) الذي يوفر التخطيط الموحد. هذه الطبقة لا تحتوي على أي منطق أعمال بل تستدعيه فقط من الطبقات السفلى.

### طبقة البنية التحتية — /infrastructure

تُعنى بإدارة الاتصالات الخارجية والخدمات الأساسية. تضم نهاية API عبر Nitro 3.0، واتصال Supabase مع 27 جدولاً محمية بـ RLS، وUpstash Redis للتخزين المؤقت الموزع، لمراقبة الأخطاء، وMixpanel لتحليلات الاستخدام. تُعزل تم منطق الأعمال.

## قاعدة التسلسل الطبقي

الاعتماديات تسير دائمًا من الأعلى إلى الأسفل: طبقة التطبيق ← طبقة الميزات ← الطبقة المشتركة ← طبقة البنية التحتية. لا يُسمح باستيراد ملف من طبقة أعلى منها. طبقة البنية التحتية هي الأدنى ولا يمكنها الاعتماد على أي طبقة أخرى. الطبقة المشتركة مسؤولة على طبقة الميزات أو التطبيق. هذا التسلسل يضمن بنية نظيفة وخالية من الاعتماديات المعقدة.

### استثناء مصرح به

يُسمح لطبقة الميزات بالاستيراد مباشرة من الطبقة المشتركة دون المرور عبر طبقة التطبيق. هذا الاستثناء مقصود لتجنب تكرار الكود وتقليل

## قواعد الاعتماديات

نظام الاعتماديات في فيكسور صُمم ليكون صارمًا وواضحًا، مما يمنع تشكل الديون التقنية ويضمن بقاء المشروع صالحًا للعمل الطويل. يُدار هذا النظام عبر قواعد محددة تُفرض برمجياً عبر أدوات تحليل ثابت للكود وتُتحقق في كل مراجعة كود `PR review`.

### القاعدة الأولى: منع الاعتماديات الدائرية

يُحظر تمامًا إنشاء حلقات اعتمادية بين أي ملفين أو نطاقين في المشروع. يُستخدم أداة `dependency-cruiser` لفحص للاعتماديات في كل `build` والتأكد من بقاءه على شكل DAG. أي دائرة اعتمادية جديدة تمنع الدمج تلقائيًا في الفرع الرئيسي هيكلية فورية.

### القاعدة الثانية: فصل العميل والخادم

جميع المكتبات التي تعمل فقط على الخادم — مثل `ccxt` للتداول، ومكتبات إرسال البريد الإلكتروني، وأدوات التشفير المتقدم فقط في ملفات تُنفذ على الخادم. يُستخدم شرط `server-only` لمنع حزم هذه المكتبات في بناء العميل. هذا يقلل حجم اللمتصفح ويحمي المفاتيح السرية من التسريب.

```
صحيح: استيراد مشروط على الخا
serverOnly } from 'vinxi/server-only';
xt = serverOnly(() => require('ccxt'));

في ملف عميل ccxt خاطئ: استير
xt from 'ccxt'; // سُرِفُض في مراجعة الكود
```

## القاعدة الثالثة: الاعتماد على واجهات التصدير فقط

عند حاجة نطاق إلى استخدام وظيفة من نطاق آخر أو من الطبقة المشتركة، يجب أن يستوردها فقط من ملف `index.ts` البرملي) وليس من الملفات الداخلية. هذا يخلق طبقة تجريد تسمح بتغيير التطبيق الداخلي لنطاق ما دون التأثير على المستهلكين

## القاعدة الرابعة: الأولوية للاعتماد على الطبقة المشتركة

عندما يحتاج نطاقان إلى نفس الوظيفة، يجب رفع هذه الوظيفة إلى الطبقة المشتركة بدلاً من جعل أحد النطاقين يعتمد على الآخر. هذا يخلق طبقة تجريد تسمح بتغيير التطبيق الداخلي لنطاق ما دون التأثير على المستهلكين. الطبقة المشتركة.

## القاعدة الخامسة: الحد الأقصى لعمق الاعتماديات

لا يجوز أن يتجاوز عمق سلسلة الاعتماديات 4 مستويات. أي سلسلة أطول تشير إلى تصميم سيء وتتطلب إعادة هيكلة. على سبيل المثال، كان مسار في طبقة التطبيق يستدعي مكوناً من طبقة الميزات، الذي يستدعي خدمة من نفس الطبقة، التي تستدعي وظيفة مشتركة، التي تستدعي خدمة من طبقة البنية التحتية — فهذا الحد الأقصى المسموح به.

### أهمية أداء

الاعتماديات الدائرية لا تؤثر فقط على قابلية الصيانة بل تسبب أيضًا مشاكل في أداء البناء `build performance`، حيث تمنع التحسين `tree-shaking` من العمل بكفاءة.

SEC-06

## الطبقة المشتركة

الطبقة المشتركة `shared/` هي العمود الفقري لنظام فيكسور، وتضم 96 ملفًا توفر البنية التحتية المشتركة لجميع النطاقات والـ تُصمم هذه الطبقة لتكون مستقرة قدر الإمكان — التغييرات فيها تؤثر على النظام بأكمله، لذا تُخضع لاختبارات صارمة قبل أي تعديل.

## الوصول إلى البيانات — /data-access

يحتوي هذا المجلد على أكثر من 26 وظيفة خادم `server function` تُعالج جميع عمليات القراءة والكتابة في قاعدة البيانات مُعلقة بنظام `RLS` الخاص بـ Supabase لضمان الأمان على مستوى الصفوف. تشمل هذه الوظائف عمليات الإدراج والتحديث والاستعلامات المعقدة، مع دعم الترقيم والتصفية والفرز. كل وظيفة تُغلف في ملف مستقل باسم وصفي يتبع نمط `create-xxx` أو `update-xxx`.

## مُوجّه نماذج اللغة الكبيرة — /llm

يدير هذا المكون الاتصال بمقدمي الذكاء الاصطناعي الأربعة: `zai` كمزود أساسي، ثم `anthropic` و `groq` و `openai` كخيارات بديلة. يعمل المُوجّه على التبديل التلقائي بين المزودين عند فشل أي منهم، مع تتبع الاستهلاك والحدود المالية لكل مزود. يدعم المُوجّه من المهام: توليد النصوص، وتحليل البيانات، والاستجابة للمحادثات.

## نظام التدويل — /i18n

يدعم اللغتين العربية والإنجليزية بالكامل. يتضمن ملفات الترجمة لكل لغة، ومزود سياق React لإدارة اللغة الحالية، وخط `useTranslation` لاستخدام الترجمات في المكونات. يتحكم تلقائيًا في اتجاه الصفحة `direction` والخطوط والتنسيقات



المختارة. تُحفظ اللغة المفضلة للمستخدم في قاعدة البيانات وتُستعاد تلقائيًا عند تسجيل الدخول.

## نظام الأحداث — /events

نظام أحداث مُطبَّق بدقة يضم أكثر من 20 حدثًا مُعرَّفًا. كل حدث له اسم فريد ونوع TypeScript محدد يَدقِّق يصف بي `EventOrchestrator` لنشر الأحداث والاشتراك فيها عبر أجزاء النظام المختلفة. من الأمثلة على الأحداث: اكتمال تحليل، وتحديث بيانات السوق، وتلقي إشعار. يضمن النظام أن كل حدث يُعالج بالترتيب الصحيح وبالتزامن عند الحاجة.

## ذاكرة التطبيق — /memory

نظام ذاكرة قائم على PostgreSQL يُستخدم لحفظ جلسات المحادثة مع الرفيق الذكي وتاريخ تفاعلات المستخدم مع النظام واسترجاع البيانات المهيكلة مع تحديد صلاحية زمنية للبيانات المؤقتة. تُكتب جميع عمليات الذاكرة عبر وظائف الوصول إلى `data-access/`.

## سجل الأدوات — /tool-registry

يُسجل جميع الأدوات المتاحة لمُوجِّه الذكاء الاصطناعي. كل أداة لها اسم وتوصيف ونوع مدخلات ومخرجات محدد بدقة. يُستخدم من قبل الرفيق الذكي `copilot` ومساعد `MOXI` لاتخاذ قرارات حول الأدوات المناسبة لكل طلب. يدعم إضافة أدوات جديدة دون تعديل الكود الأساسي.

## بيانات السوق — /market-data

يوفر بيانات السوق الحية عبر اتصاليين WebSocket مستمرين: `BinanceWS` لبيانات العملات الرئيسية، و `ScreenerWS` العملات الميمية على شبكة سولانا. يتضمن نظام إعادة اتصال تلقائي عند انقطاع الاتصال، وتخزين مؤقت للبيانات الأخيرة لتقليل وتطبيع للبيانات القادمة من مصادر مختلفة لتنسيق موحد.

## نظام المرونة — /resilience

يضم ثلاثة أنماط أساسية: قاطع الدائرة `Circuit Breaker` يوقف الطلبات إلى خدمة فاشلة بعد عدد محدد من المحاولات، و `Rate Limiter` يتحكم في عدد الطلبات المُرسلة لكل خدمة، وذاكرة `LRU` `Least Recently Used Cache` تحتفظ بأداء وتجاهل الأقدم تلقائيًا. تعمل هذه الأنماط معًا لحماية النظام من التحميل الزائد وفشل الخدمات الخارجية.

## الإشعارات — /notifications

نظام إشعارات متعدد القنوات يدعم: البريد الإلكتروني، ورسائل Telegram، وWebhooks المخصصة، والإشعارات داخل التطبيق. موحدة لإرسال الإشعارات عبر أي قناة، مع إمكانية تحديد أولوية كل إشعار وقنوات التسليم المفضلة للمستخدم. يُسجل كل إشعار حالته في قاعدة البيانات.

## التشفير — /crypto

يُوفر وظائف تشفير وفك تشفير للاعتمادات الحساسة مثل مفاتيح API الخاصة بالتداول. يُستخدم التشفير المتماثل `ES-256` مُدارة بشكل آمن. لا تُخزن أي بيانات حساسة كنص صريح في قاعدة البيانات. كل عملية تشفير تُسجل في سجل التدقيق لمتابعة البيانات الحساسة ومتى.

### 🔥 ملاحظة للمطورين

عند إضافة ملف جديد إلى الطبقة المشتركة، تأكد من إضافته إلى ملف التصدير البرملي المناسب ووثِّقه في هذا الكتاب. تغييرات الطبقة الم تتطلب مراجعة من مطورين اثنين على الأقل.

## طبقة الميزات

طبقة الميزات `features/` هي المكان الذي يعيش فيه منطق الأعمال لنظام فيكسور. كل وحدة نطاقية في هذه الطبقة تُمثل مستقل بتنظيم داخلي متسق. هذه الطبقة تضم 22 نطاقًا إجمالي مئات الملفات التي تشكل قلب النظام التشغيلي.

### البنية الداخلية لكل نطاق

يتبع كل نطاق نمطًا داخليًا موحدًا يسهل الانتقال بين النطاقات المختلفة. كل نطاق يحتوي عادةً على المجلدات التالية: `s/` للمكونات المرئية الخاصة بالنطاق، و `hooks/` للهوكس المخصصة التي تُغلف منطق النطاق، و `services/` لخدمات النطاق بال خادم والخدمات الخارجية، و `types/` لأنواع TypeScript الخاصة بالنطاق، و `utils/` للوظائف المساعدة الداخلية، و `x.ts` تصدير برملي.

### النطاقات الرئيسية وتفاصيلها

|                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>arbitrage — المراجعة</b></p> <p>ثاني أكبر نطاق بـ 27 ملفًا. يفحص فرق الأسعار بين المنصات المختلفة ويحسب فرص المراجعة المحتملة مع احتساب رسوم والانزلاق السعري. يدعم المراجعة بين DEXs و CEXs مع مراعاة مستمرة للفرص.</p>    | <p><b>analysis — التحليل</b></p> <p>أكبر نطاق في النظام بأكثر من 24 ملفًا و 9,924 سطرًا برمجيًا. يضم محرك التحليل الفني المبني على منهجية SMC/ICT. يشمل كشف مناطق السيولة، وتحليل هيكل السوق، وتحديد أوامر الصناعي، وتحليل نقاط الفشل، وتحليل أنماط الشموع. يعمل هذا المحرك على بيانات OHLCV في الوقت الفعلي.</p> |
| <p><b>copilot — الرفيق الذكي</b></p> <p>نظام متعدد الوكلاء بـ 13 ملفًا. يتكون من وكلاء متخصصين التحليل، ووكيل التداول، ووكيل المخاطر. ينسقون معًا لتقديم شاملة مدعومة بالبيانات. يستخدم نظام الذاكرة المشترك لتحديث المحادثة.</p> | <p><b>discovery — اكتشاف العملات</b></p> <p>محرك اكتشاف متقدم بـ 16 ملفًا. يفحص السوق باستمرار للعثور على عملات ميمية جديدة وواعدة على شبكة سولانا. يستخدم مؤشرات متعددة مثل حجم التداول الأولي، وعدد حاملي المحافظ، ونشاط المطورين، وبيانات الشبكة الاجتماعية.</p>                                               |

### نطاقات الدعم والتشغيل

بالإضافة إلى النطاقات الرئيسية، يضم النظام نطاقات دعم أساسية: `wallet` (16 ملفًا) لإدارة محافظ Web3 مع دعم شيفرة `trading` (15 ملفًا) لتنفيذ الصفقات وإدارتها، و `chart-intelligence` و `chart-truth` (5 ملفات لكل منهما) لتحليل الرسوم البيانية بالذكاء الاصطناعي، و `risk-governor` (4 ملفات) لحساب وإدارة المخاطر، و `paper-trading` (4 ملفات) للتجربى بدون مخاطر مالية حقيقية.

### نطاقات مساعدة

تشمل النطاقات المساعدة: `backtest` (9 ملفات) لاختبار الاستراتيجيات على بيانات تاريخية، و `debate` (7 ملفات) للمناقشة بين استراتيجيات مختلفة، و `moxi` (8 ملفات) لمساعد MOXI الذكي المخصص، و `strategy` (6 ملفات) لإدارة الاستراتيجيات، و `experiment` (5 ملفات) لإدارة التجارب، و `user` (5 ملفات) لإدارة ملفات المستخدمين، ونطاقات أصغر مثل `notes` ، `broker` ، `signal-tracking` ، `trades` ، `discover` ، `daily-loop` ، و `broker` .

#### قاعدة التوحيد

كل نطاق جديد يجب أن يتبع نفس البنية الداخلية الموحدة، حتى لو كان يحتوي على ملف واحد فقط. هذا الاتساق يُسهل انتقال المطورين بين النطاقات ويُقلل وقت الإعداد.

## طبقة التطبيق

طبقة التطبيق `app/` هي نقطة التلاقي بين المستخدم وبقية النظام. تُمسك هذه الطبقة بجميع المسارات `routes` والصفحات `pages` ، وتعمل على تجميع المكونات من طبقة الميزات وتقديمها في واجهة مستخدم متماسكة ومنضّمة. `layouts`

### AppShell — الهيكل الرئيسي للتطبيق

يمثل `AppShell.tsx` الهيكل الخارجي للتطبيق بأكمله، وهو ملف ضخّم يضم 1,688 سطرًا برمجيًا. يتضمن شريط التنقل العلوي الجانبي القابل للطي، ومنطقة المحتوى الرئيسية، والشريط السفلي، ونظام الإشعارات المركزي، ولوحة التحكم في المزودات. يد أيضًا حالة التطبيق العامة مثل حالة الاتصال والوضع الحالي للواجهة. يُنفذ AppShell مرة واحدة فقط ويُغلف جميع المسارات. يتضمن تجربة مستخدم متسقة عبر جميع الصفحات.

### PageLayout — تخطيط الصفحات

يوفر `PageLayout.tsx` تخطيطًا موحدًا لجميع الصفحات الداخلية بواقع 929 سطرًا برمجيًا. يحدد هذا المكون رأس الصفحة مع الإجراءات، ومنطقة المحتوى الرئيسية مع أنماط التمرير الموحدة، وشريط الأدوات السفلي عند الحاجة. يدعم أنماطًا مختلفة من التخطيط الكامل، والتخطيط مع شريط جانبي، والتخطيط المقسم. كل صفحة جديدة يجب أن تستخدم PageLayout لضمان التناغم.

### نظام المسارات

يتضمن النظام 44 مسارًا يُغطي جميع وظائف التطبيق. يُدار نظام المسارات عبر TanStack Start الذي يوفر تحميلًا كسولًا تلقائيًا وتقسيم الكود على مستوى المسار، ومعالجة مسبقة للبيانات `data loaders` . كل مسار مرتبط بنطاق واحد رئيسي في طبقة إمكانية استهلاك مكونات من نطاقات أخرى عند الحاجة. تُنظم المسارات في مجموعات منطقية: مسارات التحليل، ومسارات المسارات المحفوظة، ومسارات الإعدادات.

### التحميل المسبق للبيانات

كل صفحة ذات محتوى ديناميكي تستخدم محمل بيانات `data loader` يُنفذ على الخادم قبل عرض الصفحة. هذا يضمن توفر التحميل الأول ويقلل من حالة التحميل على العميل. محملات البيانات تستخدم وظائف الوصول إلى البيانات من الطبقة المشتركة سياق الخادم فقط.

#### 📌 قاعدة طبقة التطبيق

لا يجوز لكتابة أي منطق أعمال في طبقة التطبيق، إذا اكتشفت حاجة لمنطق معالجة بيانات معقد، يجب نقله إلى خدمة في طبقة الميزات أو المشتركة. طبقة التطبيق هي مجرد "غراء" يربط المكونات معًا.

## طبقة البنية التحتية

طبقة البنية التحتية `infrastructure/` تُعنى بجميع الاتصالات الخارجية والخدمات الأساسية التي يعتمد عليها النظام. تُصمم لتكون قابلة للاستبدال — يمكن تبديل أي خدمة خارجية بخدمة بديلة دون التأثير على منطق الأعمال في الطبقات العليا.

### نقاط نهاية API عبر Nitro 3.0

يتضمن النظام 14 نقطة نهاية `API endpoint` مُنفذة عبر Nitro 3.0 مع إعداد Vercel. تتعامل هذه النقاط مع: عمليات `ccxt` ، والتحقق من صحة البيانات، ووظائف إدارة المستخدمين، وعمليات الويب هوك للخدمات الخارجية. يُنفذ كل `point` مستقل `handler` يتبع نمط REST مع تحقق من المدخلات وإرجاع استجابات موحدة. يتم تجميع `ccxt` كخادم فقط `ly` لتقليل حجم حزمة العميل.

### قاعدة بيانات Supabase

يستخدم النظام Supabase كقاعدة بيانات رئيسية مع 27 جدول بيانات مُنظمة حسب النطاقات. كل جدول محمي بنظام `Row Level Security – RLS` الذي يضمن أن كل مستخدم يمكنه الوصول فقط إلى بياناته الخاصة. الجداول تُستخدم من قبل المستخدمين، وإعدادات التطبيق، وسجل الصفقات، وتحليلات السوق، وجلسات المحادثة مع الذكاء الاصطناعي، وإعدادات والإشعارات، وسجل التدقيق.

## Upstash Redis

يُستخدم Redis عبر Upstash لتخزين البيانات المؤقتة الموزعة والجلسات المؤقتة ونتائج التحليلات الحديثة. يُفصل Redis تم المحلية لل خادم، مما يسمح للنظام بالعمل على عدة نسخ بدون تعارضات. يُستخدم أيضًا لمخزن المعدل `rate limiter` الطلبات المتكررة.

## Sentry لمراقبة الأخطاء

يُرسل جميع الأخطاء التي تحدث على الخادم والعميل إلى Sentry للمراقبة المركزية. يُضاف سياق إضافي لكل خطأ يشمل: معرف والنطاق الذي حدث فيه الخطأ، ومعاملات الطلب، وتتبع المكس الكامل. تُنشئ Sentry تلقائيًا إشعارات عند اكتشاف أنما الأخطاء أو زيادة معدل الأخطاء.

## Mixpanel لتحليلات الاستخدام

يتتبع Mixpanel جميع تفاعلات المستخدمين المهمة لتحليل أنماط الاستخدام وقياس أداء الميزات. يُرسل كل حدث مع خصائص نوع الجهاز واللغة المفضلة والوقت المنقضي منذ آخر تسجيل دخول والنطاق النشط. لا تُرسل أي بيانات شخصية أو مالية إلى Mixpanel بيانات الاستخدام المجهولة.

### ⚠ قاعدة الأمان

جميع مفاتيح API وأسرار الخدمات الخارجية تُخزن في متغيرات البيئة `environment variables` فقط. لا يُسمح بتخزين أي مفاتيح الكود المصدري أو في ملفات الإعدادات.

SEC - 10

# اتفاقيات التسمية

اتفاقيات التسمية المتسقة تُسهل قراءة الكود والتنقل فيه، وتُقلل الأخطاء الناتجة عن التسمية المربكة. يتبع فيكسور مجموعة صارمة في تسمية المجلدات والملفات والعناصر البرمجية المختلفة. الالتزام بهذه الاتفاقيات إلزامي لجميع المطورين.

## تسمية المجلدات

- النطاقات:** تُسمى بأحرف صغيرة مفصولة بشرطة `kebab-case` ، `chart-intelligence` ، `paper-trading` ، `risk-governor` ، `signal-tracking` .
- المجلدات الفرعية:** تُسمى بأحرف صغيرة بدون فواصل: `components` ، `hooks` ، `services` ، `types` ، `utils` .
- مجلدات البنية التحتية:** تتبع `kebab-case` : `market-data` ، `tool-registry` ، `data-access` .

## تسمية الملفات

- مكونات React:** تُسمى بنمط `PascalCase.tsx` : `TradingPanel.tsx` ، `MarketOverview.tsx` ، `WalletConnect.tsx` .
- الهوكس المخصصة:** تبدأ بـ `use` متبوعًا باسم وصفي `camelCase.ts` : `useMarketData.ts` ، `useTrading.ts` ، `auth.ts` .
- الخدمات:** تُسمى `camelCase.ts` مع لاحقة `service` عند الحاجة: `marketService.ts` ، `analysisService.ts` .
- الأنواع:** تُسمى `PascalCase.types.ts` أو `camelCase.types.ts` : `Trade.types.ts` ، `market.types.ts` .
- ملفات التصدير:** دائمًا `index.ts` في كل مجلد نطاق.
- وظائف الخادم:** تتبع نمط الفعل-الاسم: `getMarketData.ts` ، `createTrade.ts` ، `updateUserSettings.ts` .

## تسمية العناصر البرمجية

- المكونات: `function TradingPanel() {}` : `PascalCase`
- الهوكس: `function useMarketData() {}` : `use` يبدأ بـ `camelCase`
- الثوابت: `DEFAULT_STALE_TIME` , `MAX_RETRY_ATTEMPTS` : `UPPER_SNAKE_CASE`
- الواجهات والأنواع: `type TradeStatus = ...` , `interface MarketData {}` : `PascalCase`
- الدوال العادية: `formatPrice()` , `calculateRisk()` : `camelCase`
- الأحداث: `EVENT_MARKET_UPDATED` , `EVENT_TRADE_EXECUTED` : `EVENT_` مع بادئة `UPPER_SNAKE_CASE`
- المتغيرات الخاصة: تبدأ بـ `_` : `_pendingRequests` , `_internalCache`

## تسمية الاختبارات

- ملفات الاختبارات: `marketService.test.ts` : `[اسم-الملف].test.ts`
- كتل الاختبار: تُسمى `describe()` بما يصف المكون/الدالة المُختبرة
- الحالات الفردية: تُسمى `it()` بصيغة الجملة الواضحة: `it('should return empty array when no trades exist')`

### ملاحظة 📌

تُفحص اتفاقيات التسمية تلقائيًا عبر قواعد ESLint المخصصة. أي مخالفة تُظهر تحذيرًا في بيئة التطوير وتمنع الدمج في مراجعة الكود.

SEC - 11

## تنظيم الكود

تنظيم الكود الجيد يُسهل الصيانة ويُسرّع عملية التطوير. يتبع فيكسور مجموعة من الأنماط المتفق عليها لتنظيم الملفات والكود. ذلك أنماط التصدير البرملي، والترتيب المنطقي للاستيرادات، وتنظيم المكونات والهوكس والخدمات.

### التصدير البرملي — Barrel Exports

كل نطاق ومجلد مشترك يُنهي بملف `index.ts` يُصدّر جميع الواجهات العامة. هذا يُبسط عمليات الاستيراد ويُخفي التفاصيل للمستهلكين. يتم ترتيب التصديرات في `index.ts` بالترتيب التالي: الأنواع والواجهات أولاً، ثم الثوابت، ثم الهوكس، ثم الخدمات.

مثال على التصدير البرملي - `res/market/index.ts`

```
// res/market/index.ts
import { MarketData, MarketStats, TickerInfo } from './types';
import { DEFAULT_PAIRS, MARKET_REFRESH_INTERVAL } from './constants';
import { useMarketData, useMarketStats, useTicker } from './hooks';
import { MarketOverview, MarketChart, TickerBar } from './components';
import { marketService } from './services';
```

### ترتيب الاستيرادات

يُرتَّب قسم الاستيرادات في كل ملف وفق ترتيب محدد صارم. يبدأ بمكتبات React والنظام، ثم مكتبات الطرف الثالث، ثم الالمشروع (الطبقة المشتركة أولاً، ثم النطاقات الأخرى، ثم الملفات المحلية). تُفصل كل مجموعة بسطر فارغ. تُضاف تعليقات توضح للاستيرادات المعقدة.

## تنظيم المكونات

كل مكون React يتبع الهيكل التالي: تعريف الأنواع أولاً، ثم تعريف المكون كدالة أساسية، ثم المكونات الفرعية إن وُجدت، ثم أنماذ يُفصل منطق المكون المعقد إلى هوكس مخصصة تُعرَّف في نفس مجلد النطاق. لا يُسمح بتعريف أكثر من مكون رئيسي واحد لنفسه.

## تنظيم الخدمات

تُنظم الخدمات ككائنات تحتوي على وظائف مرتبطة منطقياً. كل خدمة تتبع نمطاً موحداً: تعريف الأنواع، ثم ثوابت الخدمة (مثل ثم الوظائف الرئيسية، ثم الوظائف المساعدة. كل وظيفة في الخدمة مسؤولة عن مهمة واحدة فقط وتُوثق بتعليق JSDoc يوضح ومخرجاتها وسلوكها.

## تنظيم الهوكس

كل هوك مخصص يُعرَّف في ملف مستقل داخل مجلد `hooks/` للنطاق. يبدأ الملف بتعليق JSDoc يصف الغرض من الهوك والاستخدام، ثم تعريف الأنواع المرتبطة، ثم دالة الهوكس نفسها. الهوكس المعقدة التي تحتاج إلى حالة داخلية تستخدم `Reducer` عدة `useState` لمنع إعادة التصيير المتكررة.

### أفضل ممارسة

إذا تجاوز ملف ما 200 سطراً، فكر في تقسيمه. الملفات الكبيرة تُصعب المراجعة وتزيد احتمالية حدوث تعارضات عند العمل الجماعي. الاسالوحيد هو ملفات التخطيط الرئيسية مثل AppShell.

SEC - 12

## قواعد إدارة الحالة

يستخدم فيكسور أربع آليات مختلفة لإدارة الحالة، كل واحدة مخصصة لنوع معين من البيانات. فهم متى تستخدم كل آلية هو أحد كتابة كود نظيف وفعال. استخدام الآلية الخاطئة لحالة معينة يُسبب مشاكل في الأداء وسلوكيات غير متوقعة.

## TanStack Query — حالة الخادم

تُستخدم TanStack Query لجميع البيانات القادمة من الخادم. يُعرِّف `staleTime` بقيمة 30 ثانية افتراضياً، مما يعني أن طازجة لمدة 30 ثانية قبل أن تُعاد طلبها. تُستخدم مفاتيح الاستعلام `query keys` المُنسقة وفق نمط محدد: `[نطاق، نوع، سبيل المثال: ['market', 'ticker', 'SOL-USDT']]`. يوفر TanStack Query تلقائياً حالات التحميل والخطأ والتخزين المؤقت.

## Zustand — المؤشرات والمقاييس

يُستخدم Zustand لإدارة المقاييس `metrics` ومؤشرات الأداء الحية مثل أسعار العملات في الوقت الفعلي وأحجام التداول Zustand بخفة الوزن وعدم إعادة التصيير التلقائي عند عدم الحاجة. كل مخزن Zustand مُعرَّف في ملف مستقل بلاحقة ويُصدَّر عبر التصدير البرملي للنطاق.

## React Context — الحالة المشتركة

يُستخدم React Context لحالة التطبيق العميقة التي يحتاجها عدة مكونات في مستويات مختلفة من شجرة المكونات. يستخدم التدويل `i18n` وسياق المحفظة `wallet`. يُنصح بعدم إنشاء سياقات جديدة إلا عند الحاجة الفعلية، لأن كل سياق يضيف تصيير إضافية عند تغير قيمته.

يُستخدم `EventOrchestrator` كناقل أحداث مُطبَّق بدقة لنشر الأحداث والاشتراك فيها بين النطاقات المختلفة. على عكس `broadcast channel` العامة، يُطبَّق كل حدث بنوع TypeScript محدد، مما يضمن سلامة البيانات المُرسلة والمستقبلة. الأهم من ذلك، أن `EventOrchestrator` يعمل بشكل غير متزامن.

| الآلية            | نوع الحالة         | مثال                     | ملاحظات             |
|-------------------|--------------------|--------------------------|---------------------|
| TanStack Query    | بيانات الخادم      | بيانات المستخدم، الصفقات | مثال: staleTime: 30 |
| Zustand           | مؤشرات حية         | أسعار الوقت الفعلي       | تحديثات متكررة      |
| React Context     | حالة عميقة مشتركة  | اللغة، المحفظة           | تقليل عدد السياقات  |
| EventOrchestrator | أحداث بين النطاقات | تنفيذ صفقة               | أنواع مُطبَّقة      |

⚠ قاعدة ذهبية

لا تُخزن بيانات الخادم في Zustand أو React Context. استخدم دائماً TanStack Query لبيانات الخادم. البيانات المتكررة والسريعة مثل `Zustand` فقط هي التي تذهب إلى `Zustand`.

SEC - 13

## معايير واجهات البرمجة

تُدار جميع نقاط نهاية API في فيكسور عبر Nitro 3.0 الذي يعمل كخادم تطبيقات عالي الأداء. تتبع هذه النقاط معايير REST صارمة من المدخلات واستجابات موحدة، مما يضمن تجربة متنسقة لجميع المستهلكين سواء كانوا مكونات React أو تطبيقات خارجية.

### اصطلاحات REST

- GET** — لجلب البيانات: `/api/market/tickers` ، `/api/trades?page=1&limit=20`
- POST** — لإنشاء موارد جديدة: `/api/trades/execute` ، `/api/analysis/start`
- PUT** — لتحديث الموارد بالكامل: `/api/user/settings`
- PATCH** — لتحديث جزئي: `/api/wallet/preferences`
- DELETE** — لحذف الموارد: `/api/notifications/:id`

### بنية نقاط النهاية

كل نقطة نهاية تُعرَّف في ملف مستقل داخل مجلد `infrastructure/api/`. يتبع كل معالج `handler` نمطاً موحداً للمعاملات والتحقق منها، ثم تنفيذ العملية عبر وظائف الوصول إلى البيانات، ثم إرجاع استجابة موحدة. الأخطاء تُلتقط وتُحوَّل عبر HTTP مناسبة مع رسائل خطأ واضحة.

### التحقق من المدخلات

جميع المدخلات من العميل تُتَّحَقَّق قبل المعالجة. يُستخدم Zod لتحديد أنماط التحقق `schemas` التي تُربط بأنواع TypeScript. نهاية تحدد مخطط التحقق الخاص بها، وفشل التحقق يُرجع خطأ `400 Bad Request` مع تفاصيل الحقول الخاطئة.

### تنسيق الاستجابات

تتبع جميع الاستجابات الناجحة تنسيقاً موحداً يُغلَّف البيانات مع معلومات وصفية. الاستجابات غير الناجحة تتضمن رمز الخطأ و `ApiResponse<T>` المُعرَّف في أنواع البنية التحتية. Sentry. يُستخدم نمط الاستجابة الموحد

تنسيق الاستجابة

```
ApiResponse {
```

```
is: boolean;
```

```
type;
```

```
{
```

```
message: string;
```

```
payload: string;
```

```
payload?: Record;
```

```
payloadId: string;
```

```
{
```

```
id?: number;
```

```
type?: number;
```

```
type?: number;
```

## قاعدة الخادم فقط

جميع نقاط نهاية API تُنفذ على الخادم فقط عبر Nitro. لا يُسمح بإنشاء نقاط نهاية عميل. المكتبات الحساسة مثل CCxt تُستورد فقط في المعالجات الخادمية مع شرط server-only.

SEC - 14

## معايير الخدمات

الخدمات النطاقية `domain services` هي الوحدات التي تُغلف منطق الأعمال المعقد وتُدير التفاعلات مع الخدمات الخادمية الوصول إلى البيانات. كل خدمة تابعة لنطاق معين وتُعرّف داخل مجلد `services/` الخاص بذلك النطاق.

### بنية الخدمة القياسية

تتبع كل خدمة النمط التالي: تعريف الأنواع الداخلية أولاً، ثم ثوابت التهيئة (مثل عناوين URL وفترات المهلة)، ثم الوظائف الرئيسة للوظائف المساعدة الخاصة. تُصدّر الوظائف العامة فقط عبر ملف `index.ts` للنطاق. لا تحتوي الخدمات على حالة متغيرة تتلقى مدخلاتها وتُرجع مخرجاتها دون الاعتماد على حالة مشتركة.

### أنواع الخدمات

- خدمات البيانات:** تُغلف استدعاءات ووظائف الوصول إلى البيانات وتُحول البيانات الخام إلى أنواع منطقية للنطاق. مثال: `analysisService.ts` يحول بيانات OHLCV الخام إلى تحليل SMC مهيكل.
- خدمات التكامل:** تتصل بالخدمات الخارجية مثل منصات التداول ومقدمي الذكاء الاصطناعي. مثال: `tradingService.ts` يُنفذ `.ccxt`.
- خدمات الحساب:** تُنفذ حسابات معقدة مثل حساب المخاطر وتحليل الربحية. مثال: `riskService.ts` يحسب حجم الصفقة المناسبة على تحمل المخاطر.
- خدمات التنسيق:** تُنسّق بين خدمات متعددة لتنفيذ عمليات مركبة. مثال: `copilotService.ts` ينسق بين خدمة التحليل وخدمة المخاطر.

### معالجة الأخطاء في الخدمات

كل وظيفة في الخدمة تُغلف في كتلة `try-catch` تُطبع الأخطاء وتُعيدّها بشكل موحد. لا تُطلق الخدمات استثناءات غير مُعالجة ذلك تُرجع كائن خطأ مُهيكل يتضمن نوع الخطأ والرسالة والسياق. هذا يسمح للمستهلكين بالتعامل مع الأخطاء بطريقة متسقة.

### اختبار الخدمات



تُختبر كل خدمة بوحدات اختبار مستقلة تُحاكي `mock` جميع التبعيات الخارجية. تُكتب الاختبارات بشكل يصف السلوك المتوقع الناجحة، وحالات الخطأ المتوقعة، والحالات الحدية. تُنفذ الاختبارات تلقائيًا في خط CI/CD ويُمنع الدمج إذا فشل أي اختبار.

## قاعدة بدون حالة

الخدمات يجب أن تكون خالية من الحالة `stateless`. إذا احتجت إلى مشاركة حالة بين وظائف الخدمة، استخدم `meter object` ينقل الحالة كمعامل. هذا يجعل الخدمات قابلة للاختبار ومتوازية في التنفيذ.

SEC - 15

## معايير المكونات

مكونات React في فيكسور تتبع معايير صارمة لضمان الاتساق والأداء وسهولة الصيانة. جميع المكونات تُبنى باستخدام مكونات كأساس، مع تخصيص الأنماط عبر 4 Tailwind CSS. يُمنع بناء مكونات أساسية من الصفر عندما يوجد بديل في shadcn/ui.

### أنواع المكونات

- مكونات الصفحة:** مكونات كبيرة تُمثل صفحة كاملة وتجمع مكونات أصغر. تُعرّف في مجلد النطاق وتُستخدم في طبقة التطبيق. مثال: `TradingPanel.tsx` يضم المخطط وأوامر الشراء والبيع وسجل الصفقات.
- مكونات الميزة:** مكونات متوسطة الحجم تُمثل ميزة واحدة. تُخزن في مجلد `components/` للنطاق. مثال: `OrderForm.tsx` للأوامر.
- مكونات واجهة المستخدم:** مكونات أساسية صغيرة تُبنى على shadcn/ui. مثال: `PriceBadge.tsx` لعرض السعر بلون يُعبّر.

### هيكل المكون القياسي

كل مكون React يتبع الهيكل التالي: تعريف واجهة الدعائم `Props interface` أولاً مع تعليقات JSDoc لكل دعمة، ثم كدالة أساسية، ثم أي مكونات فرعية مُغلقة `encapsulated`. المكونات المعقدة التي تحتوي على منطق معالجة بيانات تستخدم إلى هوك مخصص يُستهلك داخل المكون.

### قواعد الأداء

- استخدم `React.memo()` للمكونات التي تتلقى نفس الدعائم بشكل متكرر ولا تحتاج إلى إعادة تصيير.
- استخدم `useMemo()` للحسابات المكلفة داخل المكون.
- استخدم `useCallback()` لدوال معالجة الأحداث المُمررة كمكونات فرعية.
- لا تُعرّف دوال أو كائنات جديدة داخل جسم المكون الرئيسي — حركها خارجه أو استخدم `useMemo`.
- قوائم البيانات الطويلة تستخدم `react-window` أو `@tanstack/react-virtual` للعرض الافتراضي.

### قواعد إمكانية الوصول

- استخدم العناصر الدلالية: `button`، `nav`، `main`، `article`.
- أضف `aria-label` لجميع العناصر التفاعلية التي لا تحتوي على نص مرئي.
- اضبط `alt` وصفيًا لجميع الصور.
- تأكد من إمكانية التنقل بالكيورد عبر جميع العناصر التفاعلية.
- استخدم `sr-only` للنصوص المخفية المرئية المهمة لقارئات الشاشة.

### مكونات shadcn/ui

جميع مكونات واجهة المستخدم في مجلد `/src/components/ui/` موجودة مسبقًا من shadcn/ui. استخدمها دائمًا بدلاً من بناء مكونات يُسمح بتخصيص الأنماط عبر Tailwind فقط.

## معايير الهوكس

الهوكس المخصصة `custom hooks` هي الآلية الأساسية لإعادة استخدام منطق الحالة والسلوك بين المكونات في فيكس مخصص يتبع معايير صارمة لضمان الاتساق وسهولة الفهم. يُعرّف كل هوك في ملف مستقل داخل مجلد `hooks/` الخاص بالن

### تصنيف الهوكس

- **هوكس البيانات:** تُغلّف استعلامات TanStack Query وتُبسّط استخدامها. مثال: `useMarketData(pair)` يُرجع البيانات وحالة والخطأ في كائن واحد. يُعرّف كل هوك بيانات مفتاح الاستعلام ومدة الطزاجة وخيارات إعادة المحاولة.
- **هوكس السلوك:** تُغلّف تفاعلات المستخدم والمنطق الشرطي. مثال: `useTradingForm()` يُدير حالة نموذج التداول والتحقق من المدخلات وإرسال الأوامر.
- **هوكس الاشتراك:** تُدير اشتراكات WebSocket والحدث. مثال: `useTickerSubscription(pairs)` يُشترك في بيانات الأسعار ويُحدّث حالة Zustand تلقائيًا.
- **هوكس التكامل:** تُسهّل التكامل مع الخدمات الخارجية. مثال: `useWalletConnection()` يُدير اتصال المحفظة الرقمية والتحقق والسماح بالعمليات.

### قواعد كتابة الهوكس

- اسم الهوك يجب أن يصف ما يفعله بدقة، لا ما يعيده: `useWalletBalance()` أفضل من `useWallet()`.
- كل هوك يجب أن يبدأ بتعليق JSDoc يصف الغرض والمدخلات والمخرجات وسلوكه.
- هوكس البيانات يجب أن تُرجع كائنًا موحدًا يضم `data` و `isLoading` و `error` و `refetch`.
- لا تُنفذ عمليات DOM مباشرة داخل الهوكس — استخدم `useRef` و `useEffect` بالشكل الصحيح.
- هوكس الاشتراك يجب أن تُنظّف اشتراكاتها في دالة إرجاع `cleanup function` في `useEffect`.
- لا تستدعي هوكسًا داخل حلقات أو شروط — فقط في مستوى أعلى من المكون.

### التوثيق المطلوب

كل هوك مخصص يجب أن يوثق بالشكل التالي: تعليق JSDoc في أعلى الملف يصف الغرض، ووصف لكل معامل، ووصف مُرجعة، وملاحظات حول السلوك المتوقع. هذا التوثيق يظهر تلقائيًا في IntelliSense عند استخدام الهوكس من مكونات أخرى المطورين على فهم كيفية الاستخدام الصحيح.

#### 🔥 قاعدة القابلية لإعادة

إذا وجدت نفسك تنسخ نفس منطق الحالة بين مكونين، استخرج هذا المنطق إلى هوك مخصص. الهوكس هي أفضل آلية لمشاركة السلوك المكونات في React.

## معايير المزودات

مزودات السياق `Context Providers` في فيكسور تُستخدم بحذر وبشكل مُحصور لأن كل مزود يضيف طبقة إعادة تصيير على تُقيّد المزودات على الحالات العميقة التي يحتاجها عدد كبير من المكونات عبر مستويات متعددة من شجرة المكونات.

### المزودات المصرح بها

- **11nProvider:** مزود التدويل الذي يوفر اللغة الحالية ودوال الترجمة لجميع المكونات. مُعرّف في الطبقة المشتركة ويُغلّف التطبيق `AppShell`. يتحكم تلقائيًا في اتجاه الصفحة والخطوط حسب اللغة المختارة.
- **WalletProvider:** مزود المحفظة الذي يُدير حالة اتصال المحفظة الرقمية وبيانات التوازن. يوفر سياقًا مشتركًا لجميع المكونات التحقق من حالة المحفظة أو قراءة التوازن أو إرسال المعاملات.

- **QueryClientProvider**: مزود TanStack Query الذي يُهيئ إعدادات التخزين المؤقت وإعادة المحاولة وإعدادات الاتصال. مُهيئ افتراضية محسّنة لتطبيق التداول الحي.

## قواعد إنشاء مزود جديد

- **التقييم الأول:** هل تحتاج حقًا إلى مزود؟ هل يمكن تحقيق نفس الهدف بهوك مخصص أو مخزن Zustand أو TanStack Query الجواب نعم لكننا الأسئلتين، فقد لا تحتاج مزودًا.
- **تحديد النطاق:** إذا لزم إنشاء مزود، ضعه في أقرب نقطة ممكنة في شجرة المكونات — لا تُغلّف التطبيق بأكمله بمزود يُستخدم في واحدة فقط.
- **تقسيم القيمة:** إذا كان المزود يحتوي على عدة قيم مستقلة، قسّمه إلى عدة مزودات لتقليل نطاق إعادة التصيير.
- **التحسين:** استخدم `useMemo` لقيمة السياق لمنع إعادة التصيير عند كل تصيير للمزود.

## هيكل المزود القياسي

يتبع كل مزود النمط التالي: تعريف السياق مع قيمة افتراضية، ثم تعريف المزود كمكون يُدير الحالة، ثم دالة `useContext` تُغفّ واضع عند الاستخدام خارج المزود. هذا النمط يضمن أمان الاستخدام ويوفّر رسائل خطأ مفيدة عند المحاولة الخاطئة.

```
نمط المزود البسيط
Context = createContext(null);

function MyProvider({ children }: Props) {
  const value = useMemo(() => ({ /* state */ }), [deps]);
  return (
    <Context.Provider value={value}>
      {children}
    </Context.Provider>
  );
}

function useMyContext() {
  const ctx = useContext(MyContext);
  if (!ctx) throw new Error('useMyContext must be used within MyProvider');
  return ctx;
}
```

### ⚠ تحذير الأداء

كل تغيير في قيمة مزود يُعيد تصيير جميع المكونات المستهلكة له، حتى لو لم تتغير القيمة الفعلية التي يستخدمها كل مكون. قلل عدد المزود واستخدم `useMemo` دائمًا.

SEC - 18

## معالجة الأخطاء

نظام معالجة الأخطاء في فيكسور مُصمم ليكون شاملاً ومتسقًا، من التقاط الأخطاء على مستوى المكونات إلى الإبلاغ المركزي الهدف هو ضمان أن أي خطأ يُعالج بشكل مناسب دون أن يُسبب انهيارًا للنظام أو تجربة مستخدم سيئة.

## Error Boundaries — حدود الأخطاء

تُستخدم حدود الأخطاء `Error Boundaries` في React لمنع انتشار الأخطاء إلى التطبيق بأكمله. يُوضع حد أخطاء حول كل لمنع فشل صفحة واحدة من التأثير على بقية التطبيق. عند التقاط خطأ، يعرض حد الأخطاء واجهة بديلة مع رسالة وصفية وزر إرسال تُرسل جميع الأخطاء الملتقطة إلى Sentry مع سياق إضافي لتمكين التشخيص السريع.

## تسوية الأخطاء — Error Normalization

جميع الأخطاء في النظام تُحوَّل إلى تنسيق موحد `VixorError` يتضمن: رمز الخطأ `code` ، والرسالة الوصفية `message` ، `severity` (منخفضة/متوسطة/حرجة)، والسياق `context` الذي حدث فيه الخطأ، والطابع الزمني `timestamp` . هذا التيسير يُسهل التعامل مع الأخطاء في جميع طبقات النظام.

## مستويات الخطأ

| المستوى  | الوصف                     | مثال                   | الإجراء                    |
|----------|---------------------------|------------------------|----------------------------|
| LOW      | أخطاء غير حرجة            | فشل تحميل صورة         | إظهار عنصر بديل            |
| MEDIUM   | أخطاء تؤثر على ميزة       | فشل تحديث بيانات السوق | إعادة المحاولة + إشعار     |
| HIGH     | أخطاء تؤثر على تدفق العمل | فشل تنفيذ صفقة         | تنبيه فوري + Sentry        |
| CRITICAL | أخطاء تؤثر على النظام     | فشل قاعدة البيانات     | صفحة صيانة + تنبيه الطوارئ |

## معالجة الأخطاء على الخادم

جميع معالجات Nitro تُغلّف في كتلة `try-catch` موحدة تلتقط الأخطاء وتُسجلها في Sentry وتُرجع استجابة HTTP متوقعة (مثل عدم وجود بيانات) تُرجع `404` ، والأخطاء الناتجة عن مدخلات خاطئة تُرجع `400` ، والأخطاء المتعلقة بالصلاحيات والأخطاء غير المتوقعة تُرجع `500` مع معرف طلب لمتابعه.

## معالجة الأخطاء على العميل

TanStack Query يُعالج أخطاء الخادم تلقائيًا عبر حالة `error` في كل استدعاء. الهوكس المخصصة تُضيف معالجة إضافية لإشعارات `toast` للمستخدم عند فشل العمليات المهمة. الأخطاء في مكونات React تُلْتَقَط عبر حدود الأخطاء. تُسجل جميع الأخطاء أيضًا في Sentry لمتابعها.

### ⚠ قاعدة صارمة

لا يجوز تجاهل الأخطاء (catch فارغ). حتى الأخطاء "غير المهمة" يجب أن تُسجل على الأقل في وحدة التحكم. الأخطاء التي تؤثر على تجربة المستخدم يجب أن تُرسل إلى Sentry دائمًا.

SEC - 19

## التسجيل والمراقبة

نظام التسجيل والمراقبة في فيكسور يضمن رؤية كاملة في سلوك النظام في بيئة الإنتاج. يُستخدم Sentry للأخطاء و Mixpanel للاستخدام، بالإضافة إلى نظام أحداث مُطَبَّق بدقة يُسجل جميع العمليات المهمة.

## Sentry — مراقبة الأخطاء

يتكامل Sentry مع النظام على مستويين: مستوى الخادم ومستوى العميل. على الخادم، يُلتقط كل خطأ في معالجات Nitro مع المعاملات والرؤوس وبيانات المستخدم. على العميل، يُلتقط كل خطأ في مكونات React وأخطاء الشبكة وأخطاء `unhandled promise rejections` . يُرسل كل خطأ مع تتبع مكثف كامل وسياق إضافي يُسهل التشخيص.

## Mixpanel — تحليلات الاستخدام

يتتبع Mixpanel الأحداث الرئيسية لفهم سلوك المستخدمين. يُرسل كل حدث مع خصائص مستخدم وأجهزة ومهمة. لا تُرسل شخصية أو مالية — فقط أحداث مجهولة مثل: فتح صفحة التحليل، وتنفيذ صفقة، وفتح اتصال المحفظة. هذه البيانات تُستخدم لتجربة المستخدم وتحديد الميزات الأكثر استخدامًا.

## نظام الأحداث المُطَبَّق

يُسجل `EventOrchestrator` أكثر من 20 حدثًا مُطَبَّقًا بدقة، كل بنوع TypeScript محدد. من هذه الأحداث: اكتمال الـ `ers` الصفقة، وتحديث بيانات السوق، وفشل مزود الذكاء الاصطناعي، واكتمال اكتشاف عملة جديدة. يمكن إضافة مستمعين `ers` حدث لتسجيله في قاعدة البيانات أو إرسال إشعارات أو تحديث لوحات المعلومات.

## مستويات التسجيل

| المستوى | الاستخدام              | مثال                        |
|---------|------------------------|-----------------------------|
| DEBUG   | معلومات تطويرية مفصلة  | قيم المدخلات والمخرجات      |
| INFO    | أحداث التشغيل المهمة   | بدء تحليل، اكتمال صفقة      |
| WARN    | مواقف غير متوقعة       | فشل مزود AI (مع احتياط)     |
| ERROR   | أخطاء تتطلب تدخلًا     | فشل استعلام قاعدة البيانات  |
| FATAL   | أخطاء حرجة توقف النظام | فشل الاتصال بقاعدة البيانات |

## سجل التدقيق

جميع العمليات الحساسة — مثل تنفيذ الصفقات، وتغيير إعدادات الأمان، والوصول إلى بيانات مشفرة — تُسجل في `audit log` دائم. كل سجل يتضمن: معرف المستخدم، ونوع العملية، والطابع الزمني، وعنوان IP، والنتيجة. لا يمكن حذف أو تدمير السجلات. التدقيق لضمان الشفافية والمساءلة.

### إرشادات التسجيل

سُجِّل كل عملية تغيير الحالة (state change) مع القيمة القديمة والجديدة. سُجِّل كل عملية خارجية (API call) مع وقت التنفيذ والنتيجة. لا تُسجِّل بيانات حساسة مثل المفاتيح أو كلمات المرور في أي مستوى.

SEC - 20

## قواعد الأداء

الأداء عامل حاسم في تطبيقات التداول حيث التأخير قد يعني خسارة مالية حقيقية. يتبع فيكسور مجموعة من القواعد لضمان الأداء وتجربة مستخدم سلسة حتى مع حجم بيانات كبير.

## التحميل الكسول — Lazy Loading

يُستخدم التحميل الكسول على مستوى المسار عبر TanStack Start، حيث يُحمَّل كود كل صفحة فقط عند التنقل إليها لأول مرة. هذا يقلل حجم الحزمة الأولية للصفحة الأولى. مثل محركات التحليل ومخططات الأسعار — تُحمَّل بشكل كسول داخل الصفحات أيضًا. هذا يقلل حجم الحزمة الأولية للصفحة الأولى.

## القوائم الافتراضية — Virtual Lists

جميع القوائم التي قد تحتوي على أكثر من 50 عنصرًا يجب أن تستخدم عرضًا افتراضيًا `virtual scrolling`. هذا يشمل: سجل الصفقات، وقائمة العملات المكتشفة، وسجل الإشعارات، وقائمة العملات المعرضة. هذا يقلل الحمل على الذاكرة. هذا يشمل: سجل الصفقات، وقائمة العملات المكتشفة، وسجل الإشعارات، وقائمة العملات المعرضة. هذا يقلل الحمل على الذاكرة.

## تقسيم الكود — Code Splitting

يتم تقسيم الكود تلقائيًا على مستوى المسار عبر TanStack Start، وعلى مستوى المكونات الثقيلة عبر `React.lazy()`. هذا يضمن أن كل صفحة تحمل فقط الكود الذي تحتاجه، مما يقلل وقت التحميل. مثل مكتبات الرسوم البيانية تُفصل في حزم منفصلة. هذا يضمن أن كل صفحة تحمل فقط الكود الذي تحتاجه، مما يقلل وقت التحميل. ملحوظ.

## تحسين الاستعلامات

- **staleTime مناسب:** تُعيَّن فترة الطزاجة لكل استعلام حسب طبيعة البيانات: 30 ثانية للبيانات شبه الثابتة، و5 ثوانٍ لبيانات الأُس للبيانات الحساسة للمستخدم.
- **التحديث الانتقائي:** عند تحديث بيانات، يُعاد جلب الاستعلام ذي الصلة فقط عبر `invalidateQueries` بدلاً من تحديث جميع الاستعلامات.
- **الترقيم:** جميع القوائم الطويلة تستخدم ترقيم الصفحات `pagination` بدلاً من تحميل كل البيانات دفعة واحدة.
- **التحميل المسبق:** بيانات الصفحات المحتملة التالية تُحمَّل مسبقاً عند تحريك المؤشر فوق روابط التنقل.

## WebSocket تحسين

اتصالات WebSocket لبيانات السوق تُدارة بعناية لتقليل الضغط. تُستخدم تقنية `debounce` لتقليل عدد التحديثات المعروفة ببيانات متكررة. البيانات تُخزن مؤقتاً في مخزن Zustand وذاكرة LRU لمنع الطلبات المتكررة. عند فقدان الاتصال، يُحاول النظام تلقائياً مع زيادة تدريجية في الفترة بين المحاولات.

### ⚡ قاعدة الأداء الحرجة

أي عملية حساسية تستغرق أكثر من 16 مللي ثانية (إطار واحد) يجب أن تُنقل إلى Web Worker أو تُنفذ على الخادم. واجهة المستخدم يجب أن تستجيب دائماً بغض النظر عن تعقيد العمليات في الخلفية.

SEC - 21

## إرشادات التوسع

يُصمم فيكسور ليتعامل مع نمو مستمر في عدد المستخدمين وحجم البيانات وتعقيد الميزات. هذه الإرشادات تضمن أن البنية دون إعادة هيكلة كبيرة.

## التوسع الأفقي

يعتمد النظام على استضافة بدون خوادم `serverless` عبر Vercel، مما يسمح بتوسع تلقائي حسب الطلب. حالة التطبيق خدمات خارجية (Redis و PostgreSQL) بدلاً من الذاكرة المحلية، مما يسمح بتشغيل عدة نسخ من التطبيق بدون تعارض. WebSocket تُدار عبر بروتوكولات مناسبة للاستضافة بدون خوادم.

## إضافة نطاق جديد

- إنشاء مجلد النطاق في `/features` باسم يتبع اتفاقية `kebab-case`.
- إنشاء المجلدات الفرعية: `utils`، `types`، `services`، `hooks`، `/components/`.
- تعريف الأنواع في ملف `types/index.types.ts`.
- بناء المكونات في `/components` مع استخدام مكونات `shadcn/ui`.
- إنشاء هوك بيانات في `hooks` يُغلف استعلامات TanStack Query.
- إنشاء خدمة في `services` للعمليات المعقدة.
- إنشاء ملف `index.ts` تصدير برملي.
- إضافة مسار جديد في طبقة التطبيق يستهلك مكونات النطاق.
- تحديث رسم الاعتماديات وتحقق من عدم وجود دوران.

## إضافة جدول بيانات جديد

- تُضاف الجداول في Supabase مع تمكين RLS على كل جدول.
- تُنشأ وظائف الوصول إلى البيانات المقابلة في `shared/data-access/`.
- تُعرَّف أنواع TypeScript مطابقة لهيكل الجدول في النطاق المعني.
- تُكتب اختبارات وحدة لوظائف الوصول للتحقق من صحة عمليات القراءة والكتابة.

## إضافة مزود ذكاء اصطناعي جديد

لإضافة مزود جديد لنماذج اللغة الكبيرة، أضف تكوين المزود في `shared/llm/` مع تحديد نقطة النهاية ونموذج المصادقة واضف المزود إلى سلسلة الاحتياط في الترتيب المطلوب. كل مزود يجب أن يُنفذ واجهة موحدة تتضمن: `generate()` ، و `getUsage()`.

## إضافة قناة إشعارات جديدة

لإضافة قناة إشعارات جديدة (مثل Discord أو Slack)، أنشئ مُرسلاً `sender` جديد في `shared/notifications/` الموحدة. أضف القناة إلى تكوين المستخدم في قاعدة البيانات وواجهة إعدادات الإشعارات. أضف التحقق من إرسال الإشعارات بنجاح.

### 📌 إرشادات عامة للتوسع

كل ميزة جديدة يجب أن تُقيّم من حيث تأثيرها على الأداء والأمان قبل التطوير. يُنصح بالبدء بتنفيذ مصغر (MVP) واختباره على بيئة تجريبية الكامل.

SEC - 2.2

## قواعد المؤسسة

قواعد المؤسسة `Enterprise Rules` هي مجموعة من المعايير والسياسات التي تضمن جودة الكود على المستوى التنظيمي. مُلزمة لجميع أعضاء فريق التطوير وتُفرض عبر أدوات آلية وعمليات مراجعة الكود.

### معايير مراجعة الكود — Code Review

- كل طلب دمج `PR` يتطلب موافقة مطورين اثنين على الأقل قبل الدمج في الفرع الرئيسي.
- تُفحص جميع التغييرات آليًا عبر ESLint و TypeScript compiler قبل قبول الطلب.
- مراجعات الكود تركز على: صحة المنطق، والالتزام بالمعايير المُعرّفة في هذا الكتاب، وأمان البيانات، والأداء.
- التغييرات على الطبقة المشتركة تتطلب مراجعة إضافية من قائد الفريق التقني.
- لا يُسمح بدمج كود يحتوي على تعليقات `TODO` دون إنشاء issue مُوافق عليه في نظام تتبع المهام.

### إدارة الديون التقنية

تُسجل جميع الديون التقنية كـ issues في نظام التتبع مع تصنيف `tech-debt`. كل دين تقني يُقيّم من حيث التأثير على الأمان ومدى الصعوبة في الإصلاح. يُخصص 20% من كل دورة تطوير لسداد الديون التقنية المُتراكمة. الأولوية للأديان ذات التأثير الأعلى الأمان.

### معايير التوثيق

- كل ملف خدمة أو هوك مخصص يجب أن يتضمن تعليق JSDoc في أعلاه يصف الغرض والاستخدام.
- كل وظيفة عامة يجب أن تُوثّق مع وصف المدخلات والمخرجات والسلوك المتوقع.
- تُحدّث هذه الوثيقة (كتاب البنية) مع كل تغيير جوهري في البنية.
- يُكتب README لكل نطاق يصف مسؤولياته وواجهاته واعتمادياته.

### معايير الأمان

- جميع الاعتمادات الحساسة تُخزن مشفرة في قاعدة البيانات عبر `shared/crypto/`.
- مفاتيح API وأسرار الخدمات تُحفظ فقط في متغيرات البيئة — لا تُوجد أبدًا في الكود المصدري.
- يُفقل RLS على كل جدول في Supabase، حتى الجداول المؤقتة.
- جميع المدخلات من المستخدم تُتحقق على الخادم قبل المعالجة، بغض النظر عن التحقق على العميل.

- يُجرى فحص أمني شامل ربع سنوي لجميع التبعيات والخدمات.

## سياسة الإصدارات

يتبع فيكسور نظام الإصدارات الدلالية **Semantic Versioning**. تغييرات البنية الرئيسية (مثل إعادة هيكلة النطاقات) تُ increment الرئيسي. إضافة ميزات جديدة تُ increment الرقم الفرعي. إصلاحات الأخطاء تُ increment رقم التصحيح. كل إصدار يُرفقه مفصل يصف جميع التغييرات والإضافات والإصلاحات.

## سياسة التوظيف والتدريب

كل مطور جديد يخضع لفترة تدريب تتضمن: دراسة هذا الكتاب بالكامل، واستكشاف البنية الفعلية للمشروع، وإكمال مهمة تطوير يُعيّن لكل مطور جديد مرشد **mentor** من الفريق الحالي يُرافقه خلال الأسابيع الثمانية الأولى. يُقيّم المطور الجديد بعد الفترة من فهمه للبنية والمعايير.

### ✓ الالتزام المؤسسي

الالتزام بهذه القواعد ليس خيارًا — إنه شرط أساسي للعمل على مشروع فيكسور. الجودة لا تُحدث بالصدقة بل بالالتزام واعٍ وصارم بالمعايير عليها. كل سطر كود نكتبه يعكس مستوى احترافيتنا.



وثيقة رسمية لمنظومة فيكسور الهندسية

VIXOR — AI-Powered Trading Terminal